

Lab Assignment 3

ECE 253 - FA25

November 25, 2025

Problem 1: Histogram Equalization

1. Read the image `flower.png` provided.
2. **Global Histogram Equalization:**
 - Write a function `global_histogram_equalize(image)` that performs histogram equalization on the entire image. Your function should:
 - Compute the histogram and cumulative distribution function (CDF) of the grayscale image.
 - Normalize the CDF and map the pixel values to enhance the image contrast globally.
3. **Local Histogram Equalization:**
 - Implement local histogram equalization by dividing the image into non-overlapping blocks and applying histogram equalization on each block.
 - Write a function `local_histogram_equalize(image, block_size)` that accepts the image and block size, then equalizes the histogram of each block individually.
4. **CLAHE (Contrast Limited Adaptive Histogram Equalization):**
 - Use an existing Python module (e.g., OpenCV's CLAHE function) to apply CLAHE to the image.
5. **Plotting and Comparison:**
 - Display the original image, globally equalized image, locally equalized image, and CLAHE-enhanced image side by side.
 - For each method, compute and display the histogram.
 - Compare the visual results and histograms for global histogram equalization, local histogram equalization, and CLAHE.

Problem 2: Intensity Transformation - Gamma Correction

1. Gamma Correction:

- Write a function `gamma_correction(image, gamma)` that applies gamma correction to the image. The function should:
 - Use the formula $I_{\text{out}} = I_{\text{in}}^\gamma$, where I_{in} and I_{out} are pixel intensities scaled to the range $[0, 1]$.
 - Map the pixel values to adjust the image's brightness and contrast based on the gamma value.
- Test the function with γ values less than 1 (brightening) and greater than 1 (darkening).

2. Plotting and Analysis:

- Display the original image and gamma-corrected images for different values of gamma.
- Describe how gamma correction affects the image contrast and why it might be useful in different lighting conditions.

Problem 3: Median Filtering with Salt-and-Pepper Noise

1. Adding Salt-and-Pepper Noise:

- Write a function `add_salt_pepper_noise(image, prob)` that introduces salt-and-pepper noise into the image, with `prob` representing the noise probability.

2. Applying Median Filtering:

- Write a function `median_filter(image, kernel_size)` that applies a median filter to the noisy image.

3. Plotting and Analysis:

- Display the original, noisy, and filtered images.
- Describe the effectiveness of median filtering for salt-and-pepper noise removal and discuss any limitations observed.

Problem 4: Butterworth Notch Reject Filtering in Frequency Domain

This problem will follow Figure 4.64 in section 4.10.2 on Notch Filters from Gonzalez & Woods, 3rd Edition.

1. Image Preprocessing:

- Load the image `Car.tif` and pad it to a size of 512x512 by adding zero padding on all sides of the image.
- Apply the 2D-FFT on the padded image, compute the log magnitude, and shift the DC component to the center of the image using `fftshift`.
- Display the resulting 2D-FFT log magnitude image.

You should observe symmetric "impulse-like" bursts, which are suspected to cause the Moiré pattern (the dot pattern from the newspaper). This pattern can be removed using a Butterworth Notch Reject Filter.

2. Butterworth Notch Reject Filter:

- Define a Butterworth Notch Reject Filter $H_{NR}(u, v)$ given by:

$$H_{NR}(u, v) = \prod_{k=1}^K \left[\frac{1}{1 + \left(\frac{D_0}{D_k(u, v)} \right)^{2n}} \right] \left[\frac{1}{1 + \left(\frac{D_0}{D_{-k}(u, v)} \right)^{2n}} \right]$$

where $D_k(u, v)$ and $D_{-k}(u, v)$ are defined as:

$$D_k(u, v) = \sqrt{(u - u_k)^2 + (v - v_k)^2}$$

$$D_{-k}(u, v) = \sqrt{(u + u_k)^2 + (v + v_k)^2}$$

- Choose parameters K , n , and D_0 to control the transition and radius of the notch filters.
- The parameters u_k and v_k represent the location of each impulse-like burst in the 2D DFT magnitude image. Use the following grid for all possible (u, v) coordinate pairs to generate H_{NR} :

```
x_axis = np.linspace(-256, 255, 512)
y_axis = np.linspace(-256, 255, 512)
[u, v] = np.meshgrid(x_axis, y_axis)
```

3. Application of the Filter:

- Apply the Butterworth Notch Reject Filter to the 2D DFT of the padded image.
- Compute the inverse 2D FFT of the filtered frequency image to obtain the final filtered image in the spatial domain.

4. Repeat for Street.png:

- Repeat the same steps with `Street.png`, but set $K = 2$ and remove the burst along the $u = 0$ axis and the $v = 0$ axis.